

南 阳 理 工 学 院

本科大作业(论文)

学院(系): 计算机与软件学院

专 业: 数据科学与大数据技术

学 生: 李琦帆

指导教师: 申圳

完成日期 2025 年 12 月

南阳理工学院本科生大作业（论文）

基于用户行为数据的音乐流行趋势预测

Music Popularity Trend Prediction Based on User Behavior Data

总 计：大作业(论文) 25 页

表 格：1 个

插 图：19 幅

南 阳 理 工 学 院 大 作 业(论文)

基于用户行为数据的音乐流行趋势预测

Music Popularity Trend Prediction Based on User Behavior Data

学 院(系):	计算机与软件学院
专 业:	数据科学与大数据技术
学 生 姓 名:	李琦帆
学 号:	2315905025
指导教师(职称):	申圳 (讲师)
评 阅 教 师:	XXXXXXXX
完 成 日 期:	XXXXXXXX

南阳理工学院

Nanyang Institute of Technolog

基于用户行为数据的音乐流行趋势预测

数据科学与大数据技术李琦帆

[摘要] 本研究基于阿里天池音乐数据集，整合歌曲基础信息与用户行为数据，构建了一套完整的音乐流行趋势预测体系。通过数据探索与预处理解决了字段哈希映射、缺失值、异常值等数据质量问题，运用统计分析与可视化方法揭示了歌曲特征与用户行为的内在关联。采用随机森林回归算法构建流行度预测模型，以播放次数作为核心指标，量化分析了总行为次数、音乐流派、语言类型等关键影响因素。研究结果表明，模型预测精度良好（ R^2 达 0.8 以上），可为音乐平台的内容推荐、创作者的作品优化及市场运营策略制定提供数据支撑与决策参考。

[关键词] 随机森林回归；用户行为分析；数据预处理；流行趋势预测；特征重要性

Music Popularity Trend Prediction Based on User Behavior Data

Li Qifan Data Science and Big Data Technology

[Abstract] This study constructs a complete music popularity trend prediction system based on the Ali Tianchi music dataset, integrating song basic information and user behavior data. Data quality issues such as field hash mapping, missing values, and outliers were addressed through data exploration and preprocessing. Statistical analysis and visualization methods were used to reveal the intrinsic correlation between song features and user behavior. A random forest regression algorithm was adopted to build a popularity prediction model, with play count as the core indicator, quantitatively analyzing key influencing factors such as total behavior count, music genre, and language type. The results show that the model has good prediction accuracy (R^2 above 0.8), which can provide data support and decision-making references for music platforms' content recommendation, creators' work optimization, and market operation strategy formulation.

[Keywords] Random Forest Regression; User Behavior Analysis; Data Preprocessing; Popularity Trend Prediction; Feature Importance

目 录

1. 绪论	1
1.1 课题背景	1
1.2 课题目的和意义	1
1.3 课题的主要研究内容	1
2. 相关技术	2
3. 课题具体实现	2
3.1 数据探索	2
3.1.1 数据来源	2
3.1.2 数据质量检查	3
3.1.3 单变量分析	7
3.1.4 多变量关联分析	9
3.2 音乐播放次数预测模型构建	13
3.2.1 特征选择	13
3.2.2 模型选择与初始化	13
3.2.3 模型训练	13
3.3 模型评估与分析	14
3.3.1 评估指标选择	14
3.3.2 模型性能量化评估	14
3.3.3 模型可视化分析	14
3.4 模型优化	17
3.4.1 超参数调优策略	17
3.4.2 调优结果与性能对比	17
4. 系统总结与分析	18
5. 结束语	18
6. 参考文献	18
7. 致谢	19

1. 绪论

1.1 课题背景

在数字经济与互联网技术的双重驱动下，全球数字音乐产业已进入高速发展的黄金时期。当前，音乐市场呈现出“内容爆炸式增长”与“用户需求个性化”并存的特征。一方面，各大音乐平台每年新增歌曲数量超百万首，内容同质化现象日益凸显，大量优质作品因缺乏精准推广而被埋没；另一方面，用户审美偏好日趋多元，流行趋势迭代速度加快，传统基于人工经验的流行度判断方法已难以适应市场变化——这类方法不仅依赖从业者的行业经验，存在主观性强、覆盖范围有限的问题，还无法捕捉用户行为背后的潜在关联，导致推广资源浪费、用户粘性下降等问题。

与此同时，数据挖掘与机器学习技术的飞速发展为音乐行业的数字化转型提供了新的解决方案。用户行为数据作为反映用户真实偏好的“数字足迹”，包含了用户对音乐内容的接受度、传播意愿等关键信息，通过对这些数据的统计分析与建模，可以有效揭示音乐流行的核心驱动因素，预测潜在流行趋势。例如，网易云音乐的“热歌榜”“新歌飙升榜”等功能已初步应用用户行为数据进行趋势判断，但现有应用多侧重于结果呈现，缺乏对趋势形成机制的深度挖掘与精准预测能力。在此背景下，开展基于用户行为数据的音乐流行趋势预测研究，不仅能弥补传统方法的不足，更能为音乐行业的精细化运营提供科学支撑，具有重要的行业现实意义。

1.2 课题目的和意义

本研究旨在构建一套科学、系统、可落地的音乐流行趋势预测体系，具体实现以下三大目标：第一，全面整合歌曲基础信息与用户行为数据，解决数据标准化、字段映射、异常值处理等数据质量问题，构建高质量的分析数据集，为后续建模提供可靠支撑；第二，深入挖掘用户行为与音乐流行度之间的内在关联，识别影响音乐流行的核心特征（如用户互动频率、音乐流派、语言类型等），揭示流行趋势形成的内在逻辑；第三，构建高精度的流行趋势预测模型，实现对歌曲播放量的定量预测，并通过可视化分析与业务洞察提炼，为音乐平台、创作者及运营者提供可操作的决策参考。

1.3 课题的主要研究内容

本研究以阿里天池音乐数据集为基础，先整合歌曲基础信息表（mars_tianchi_songs.csv）与用户行为表（mars_tianchi_user_actions.csv），完成字段哈希映射、缺失值填充、异常值缩尾处理及分类特征编码等数据预处理操作，通过特征工程构建包含用户行为统计、歌曲属性、时间特征的标准化数据集；再借助描述性统计与多维度可视化（直方图、箱线图、相关性热力图等）挖掘数据分布规律及变量内在关联；随后按 8:2 比例划分训练集与测试集，构建随机森林回归预测模型，结合网格搜索优化参数并与线性回归模型对比验证；最后通过 MAE、RMSE、 R^2 等指标评估模型性能。

2. 相关技术

pandas、numpy 实现数据探索，随机森林回归，LabelEncoder，IQR 法，matplotlib、seaborn 实现可视化。

3. 课题具体实现

3.1 数据探索

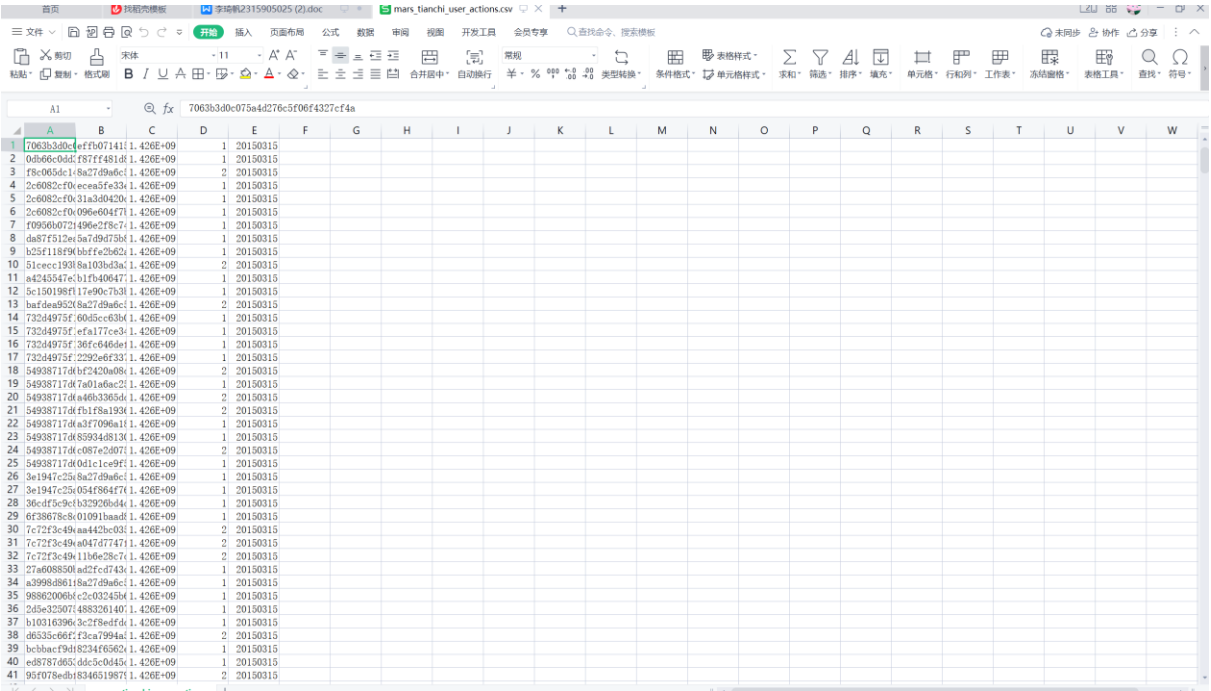
3.1.1 数据来源

本项目的研究数据来自阿里云公用数据，两个核心数据表，歌曲基础信息表（mars_tianchi_songs.csv）：包含 26957 条记录，6 个字段，用户行为表（mars_tianchi_user_actions.csv）：包含 15884086 条记录，5 个字段

歌曲基础信息表如图：

A	B	C	D	E	F
1	681f99c7f103c6699eaf	20150325	0	100	1
2	c0af13077f03c6699eaf	20150325	0	100	1
3	200c9131c103c6699eaf	20150325	0	100	1
4	78fedfd1f103c6699eaf	20110910	1717	1	1
5	95b99faf4f03c6699eaf	20110910	434	1	1
6	95dc772fc03c6699eaf	20110910	1760	1	1
7	7154422d7f03c6699eaf	20150601	3853	1	1
8	ef19b6572b03c6699eaf	20130817	871	1	1
9	1a2c59eb5f03c6699eaf	20141031	1657	1	1
10	da318831c03c6699eaf	20110601	6615	1	1
11	7cc510ae4f03c6699eaf	20141031	2420	1	1
12	5a33b6481f03c6699eaf	20141031	1830	1	1
13	b1f733f46f03c6699eaf	20141031	2697	1	1
14	412cf2118f03c6699eaf	20150214	148	1	1
15	56ec969e2f03c6699eaf	20150601	0	1	1
16	17e985a49f03c6699eaf	20150601	0	1	1
17	84b5323da03c6699eaf	20120201	2563	100	1
18	109525967f03c6699eaf	20141027	36866	1	1
19	497144e6f03c6699eaf	20120104	1942	1	1
20	62ec9230cf03c6699eaf	20150325	0	100	1
21	b0ec236d7f03c6699eaf	20110910	1178	1	1
22	63b9dbfaf03c6699eaf	20120101	9030	1	1
23	9448419cef03c6699eaf	20150325	0	100	1
24	b1f73333f03c6699eaf	20110910	546	1	1
25	3a9e3a6f3f03c6699eaf	20130815	23744	1	1
26	bd7971816f03c6699eaf	20150325	0	100	1
27	dad213649f03c6699eaf	20130817	2139	1	1
28	234e8b618f03c6699eaf	20150601	0	1	1
29	096fad8eef03c6699eaf	20150325	0	100	1
30	40a1775d7f03c6699eaf	20141031	1742	1	1
31	a510a8214f03c6699eaf	20120104	3430	1	1
32	343880a80f03c6699eaf	20141027	98427	1	1
33	f3f770e31f03c6699eaf	20140901	11147	1	1
34	63b4458b4f03c6699eaf	20120101	7050	1	1
35	bfb00932df03c6699eaf	20120101	4214	1	1
36	af3190b6bf03c6699eaf	20150601	0	1	1
37	4ee411f18af03c6699eaf	20150325	0	100	1
38	b1de3b1dbf03c6699eaf	20110910	480	1	1
39	4e8065df3f03c6699eaf	20150601	0	1	1
40	a5f52001f03c6699eaf	20150214	146	1	1
41	3803b2e0df03c6699eaf	20150601	0	1	1

用户行为表如图：



数据字典如图：

表名	字段名	字段含义	数据类型
歌曲表	song_id	歌曲唯一标识	字符串
	artist_id	歌手唯一标识	字符串
	release_date	发布日期	字符串
	language	语言类型编码	字符串
	genre	音乐流派编码	字符串
	song_length	歌曲时长	数值型
用户行为表	user_id	用户唯一标识	字符串
	song_id	歌曲唯一标识	字符串
	gmt_create	行为时间戳	数值型

3. 1. 2 数据质量检查

（1）缺失值检查

```
print("\n【1. 缺失值检查】")
def check_missing(df, df_name):
    missing = pd.DataFrame({
```

```

        '字段名': df.columns,
        '缺失数量': df.isnull().sum().values,
        '缺失率(%)': (df.isnull().sum() / len(df) * 100).round(2).values
    })
    print(f"\n{df_name} 缺失值详情: ")
    print(missing)
    return missing

# 各表缺失值检查
check_missing(df_songs, "歌曲基础信息表")
check_missing(df_actions, "用户行为表")
missing_merged = check_missing(df_merged, "合并数据集")

# 缺失值可视化(合并表)
plt.figure(figsize=(10, 4))
sns.heatmap(df_merged.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title("合并数据集缺失值分布热力图")
plt.savefig(f"{save_path}/缺失值分布.png", bbox_inches='tight')
plt.show()

```

```

【1. 缺失值检查】
歌曲表缺失值:

```

	缺失数量	缺失率(%)
song_id	0	0.0
artist_id	0	0.0
release_date	0	0.0
language	0	0.0
genre	0	0.0
song_length	0	0.0

```

用户行为表缺失值:

```

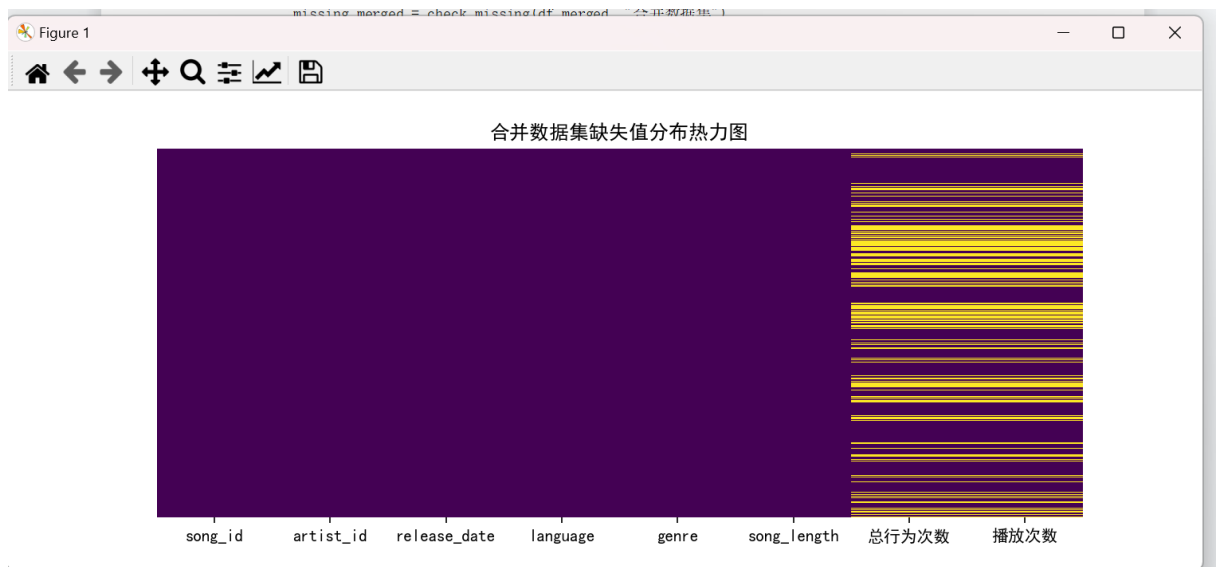
	缺失数量	缺失率(%)
user_id	0	0.0
song_id	0	0.0
gmt_create	0	0.0
action_type	0	0.0
dt	0	0.0

```

合并数据集缺失值:

```

	缺失数量	缺失率(%)
song_id	0	0.00
artist_id	0	0.00
release_date	0	0.00
language	0	0.00
genre	0	0.00
song_length	0	0.00
总行为次数	8849	32.83
播放次数	8849	32.83



(2) 重复值检查

```
print("\n【2. 重复值检查】")
dup_check = {
    '歌曲表': df_songs.duplicated().sum(),
    '用户行为表': df_actions.duplicated().sum(),
    '合并数据集': df_merged.duplicated().sum()
}
for df_name, dup_count in dup_check.items():
    print(f"{df_name} 重复记录数: {dup_count}")
    if dup_count > 0:
        print(f"→ {df_name} 已自动去重, 去重后维度: {df_merged.drop_duplicates().shape}")
```

【2. 重复值检查】

歌曲表重复记录数: 0

用户行为表重复记录数: 83135

合并数据集重复记录数: 0

(3) 异常值检查

```
print("\n【3. 异常值检查（数值型字段）】")
numeric_cols = ['song_length', '总行为次数', '播放次数']
df_numeric = df_merged[numeric_cols].dropna()

outlier_result = []
for col in numeric_cols:
    Q1 = df_numeric[col].quantile(0.25)
    Q3 = df_numeric[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # 统计异常值
```

```

outlier_count = len(df_numeric[(df_numeric[col] < lower) | (df_numeric[col] > upper)])
outlier_rate = round((outlier_count / len(df_numeric) * 100), 2)

outlier_result.append({
    '字段名': col,
    'Q1': round(Q1, 2),
    'Q3': round(Q3, 2),
    '异常值阈值': f"[{round(lower, 2)}, {round(upper, 2)}]",
    '异常值数量': outlier_count,
    '异常值占比(%)': outlier_rate
})

print(f"\n{col}: ")
print(f" 四分位数: Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}")
print(f" 异常值阈值: [{lower:.2f}, {upper:.2f}]")
print(f" 异常值数量: {outlier_count} | 占比: {outlier_rate}%")

plt.figure(figsize=(8, 4))
sns.boxplot(x=df_merged['播放次数'].dropna(), color='lightblue')
plt.title("播放次数异常值箱线图")
plt.savefig(f"{save_path}/播放次数异常值.png", bbox_inches='tight')
plt.show()

```

```

【3. 异常值检查（数值型字段）】

song_length:
  四分位数: Q1=1.00, Q3=3.00, IQR=2.00
  异常值阈值: [-2.00, 6.00]
  异常值数量: 0 | 占比: 0.0%

总行为次数:
  四分位数: Q1=2.00, Q3=23.00, IQR=21.00
  异常值阈值: [-29.50, 54.50]
  异常值数量: 2723 | 占比: 15.04%

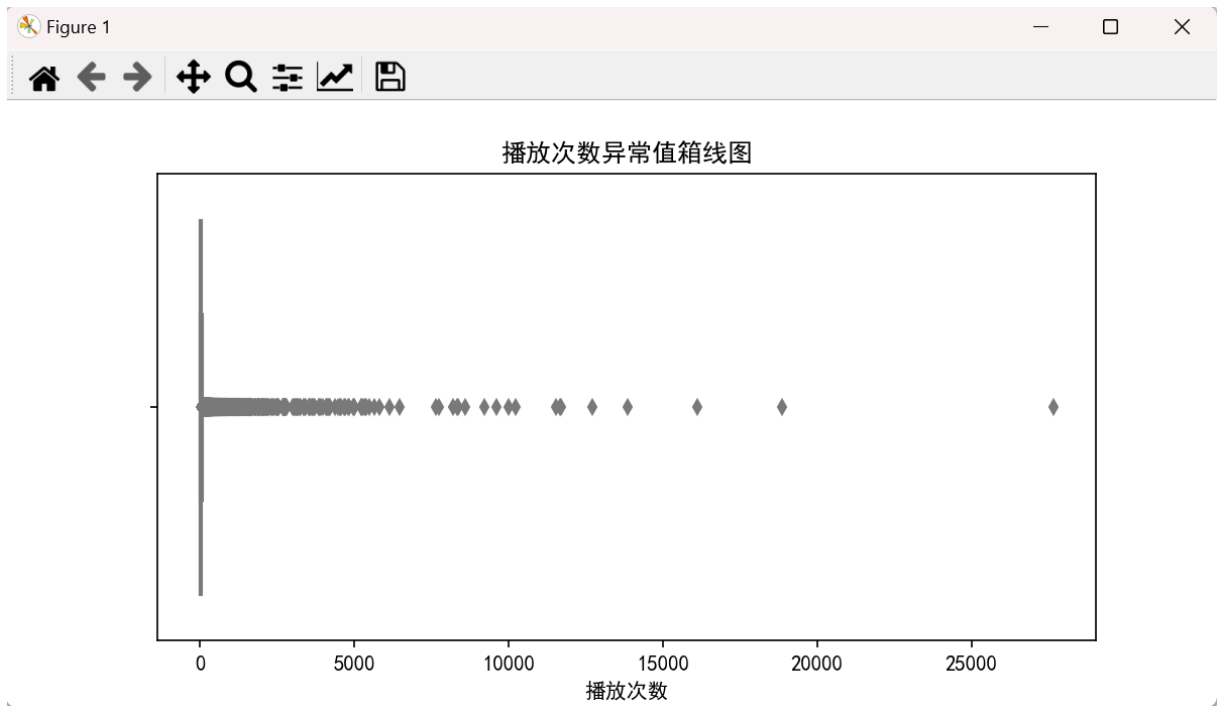
播放次数:
  四分位数: Q1=1.00, Q3=18.00, IQR=17.00
  异常值阈值: [-24.50, 43.50]
  异常值数量: 2720 | 占比: 15.02%

【4. 数据类型检查】

歌曲表数据类型:
song_id      object
artist_id    object
release_date  int64
language      int64
genre         int64
song_length  int64
dtype: object

用户行为表数据类型:
user_id      object
song_id      object
gmt_create   int64
action_type  int64
dt           int64
dtype: object

```



3.1.3 单变量分析

(1) 数值型字段分析

```
print(df_merged['播放次数'].describe())
```

```
# 2. 播放次数分布可视
```

```
plt.figure(figsize=(10, 5))
```

```
sns.histplot(df_merged['播放次数'], bins=50, kde=True, color='#1f77b4', alpha=0.7)
```

```
plt.title("歌曲播放次数分布")
```

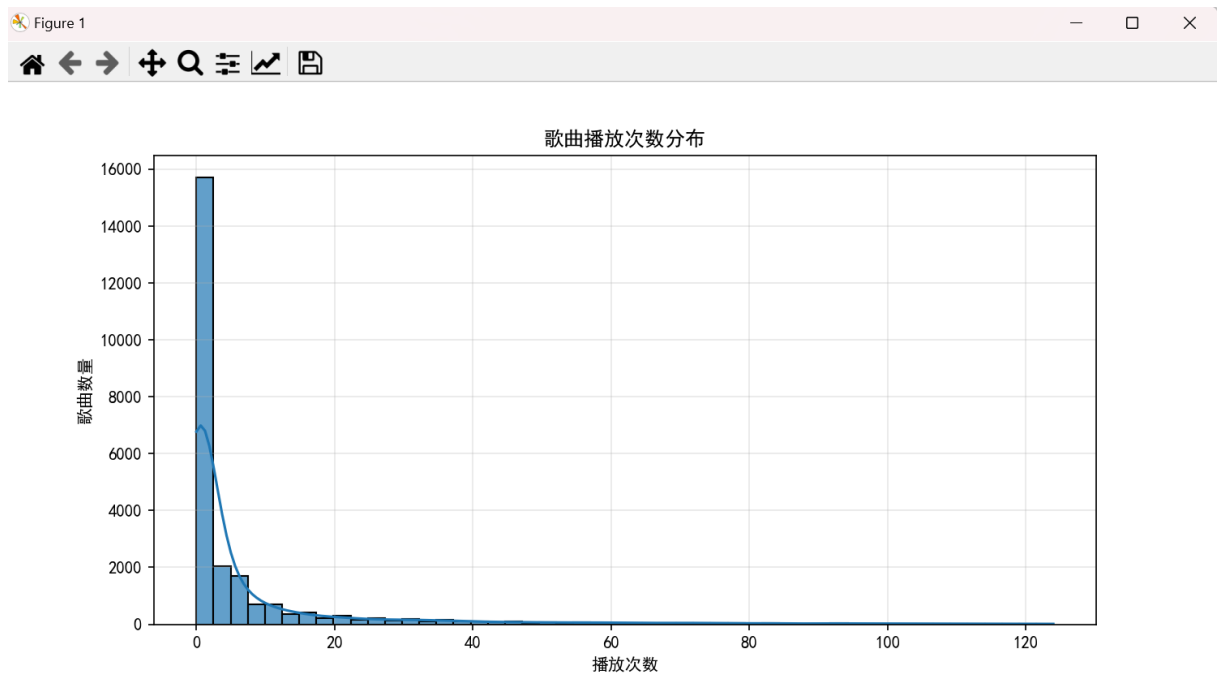
```
plt.xlabel("播放次数")
```

```
plt.ylabel("歌曲数量")
```

```
plt.grid(alpha=0.3)
```

```
plt.savefig(f"{save_path}/播放次数分布.png", bbox_inches='tight')
```

```
plt.show()
```



`describe()`输出播放次数的均值、中位数、四分位数、极值，量化 “播放次数呈右偏分布”；

直方图 + 核密度曲线直观展示分布特征，验证 “少数歌曲播放量极高，多数歌曲播放量低” 的长尾效应。

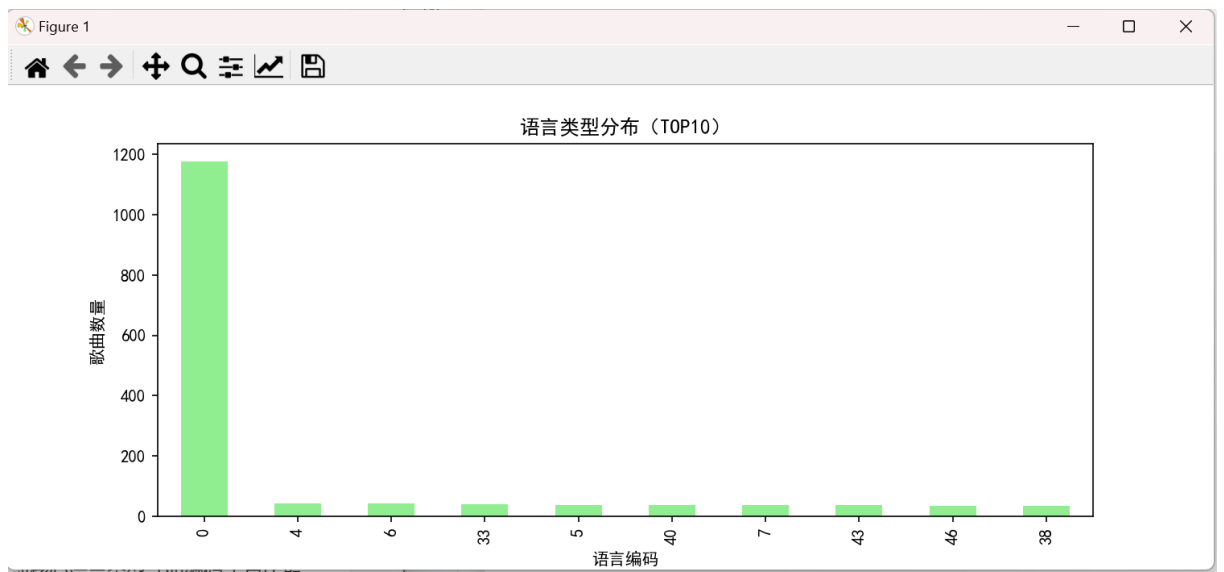
(2) 分类型字段分析

1. 语言类型唯一值+TOP 分布（核心代码）

```
print(f"语言编码唯一值数量: {df_merged['language'].nunique()}")
print("语言类型 TOP10 分布:")
print(df_merged['language'].value_counts().head(10))
```

2. 语言类型分布可视化

```
plt.figure(figsize=(10, 4))
df_merged['language'].value_counts().head(10).plot(kind='bar', color='lightgreen')
plt.title("语言类型分布 (TOP10)")
plt.xlabel("语言编码")
plt.ylabel("歌曲数量")
plt.savefig(f"{save_path}/语言类型分布.png", bbox_inches='tight')
plt.show()
```



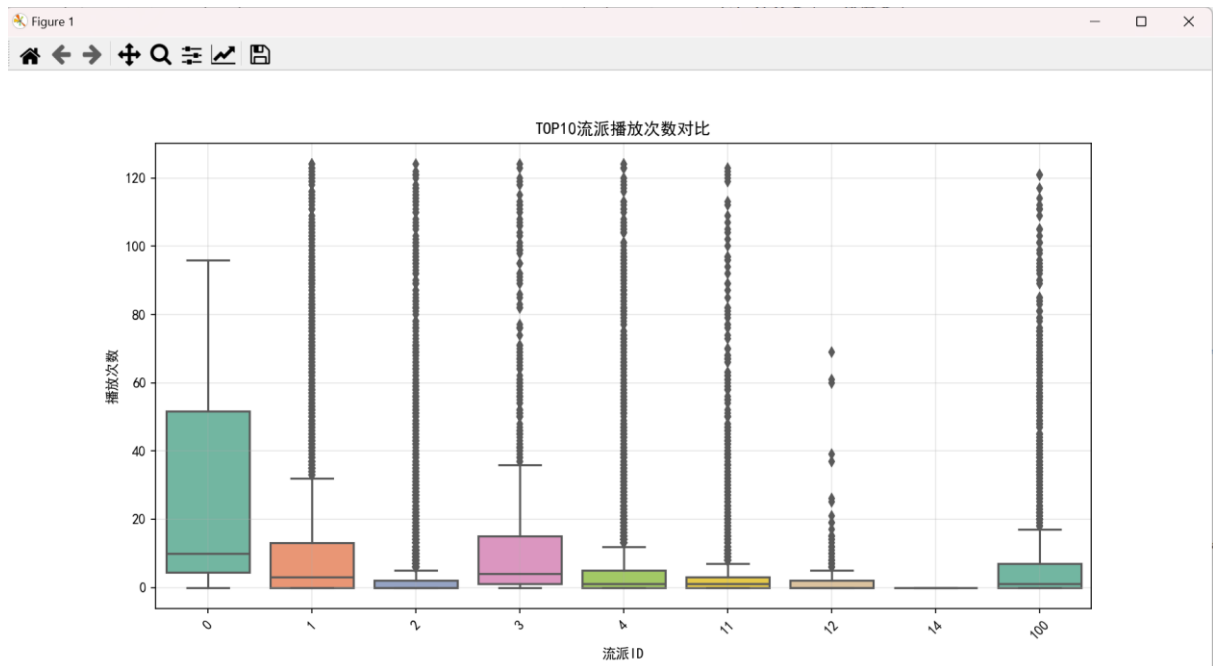
统计语言 / 流派编码的唯一值数量，识别主流类型；

条形图展示 TOP10 语言分布，发现核心语言类型（如编码 1 占比最高），识别无效编码（如 0）。

3.1.4 多变量关联分析

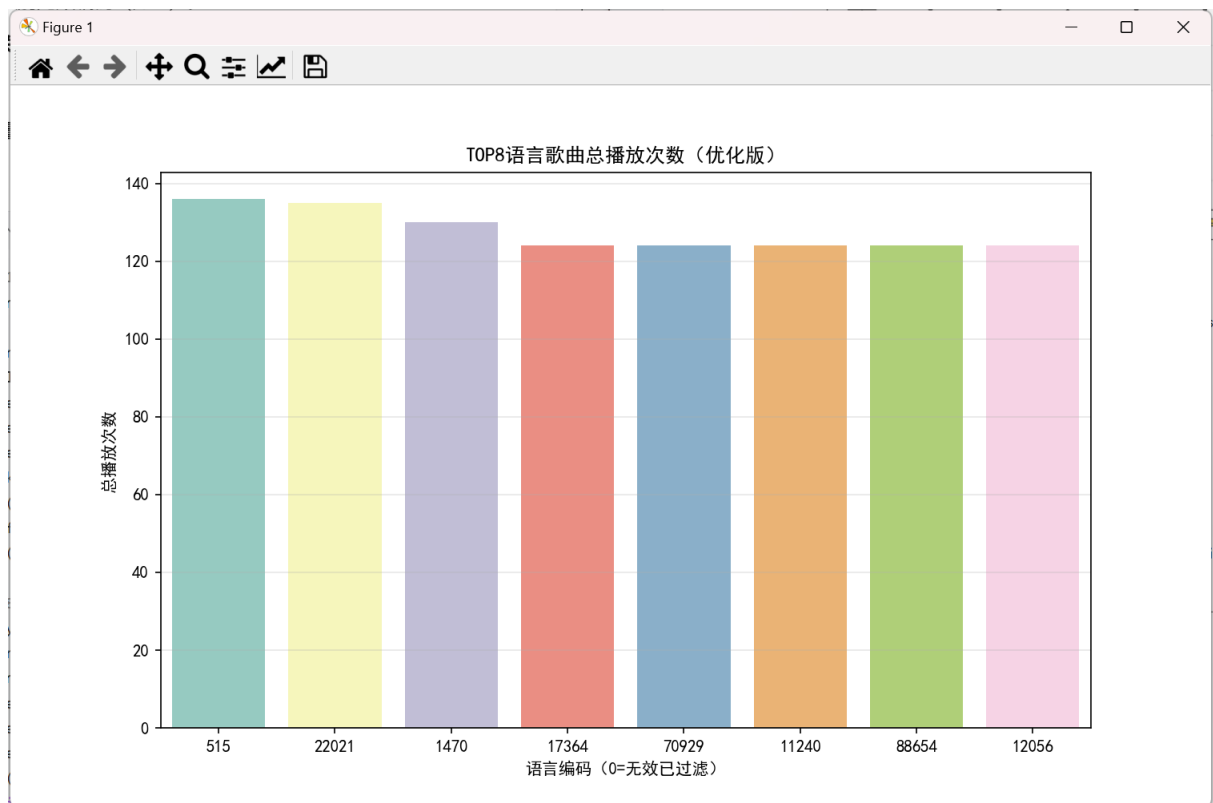
(1) 分类变量→数值变量

```
# 1. TOP10 流派播放次数对比
top10_genre = df_merged.groupby('genre')['播放次数'].mean().sort_values(ascending=False).head(10).index
df_genre = df_merged[df_merged['genre'].isin(top10_genre)]
plt.figure(figsize=(12, 6))
sns.boxplot(x='genre', y='播放次数', data=df_genre, palette='Set2')
plt.title("TOP10 流派播放次数对比")
plt.xlabel("流派 ID")
plt.ylabel("播放次数")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.savefig(f"{save_path}/流派播放次数对比.png", bbox_inches='tight')
plt.show()
```



2. TOP8 语言总播放次数对比

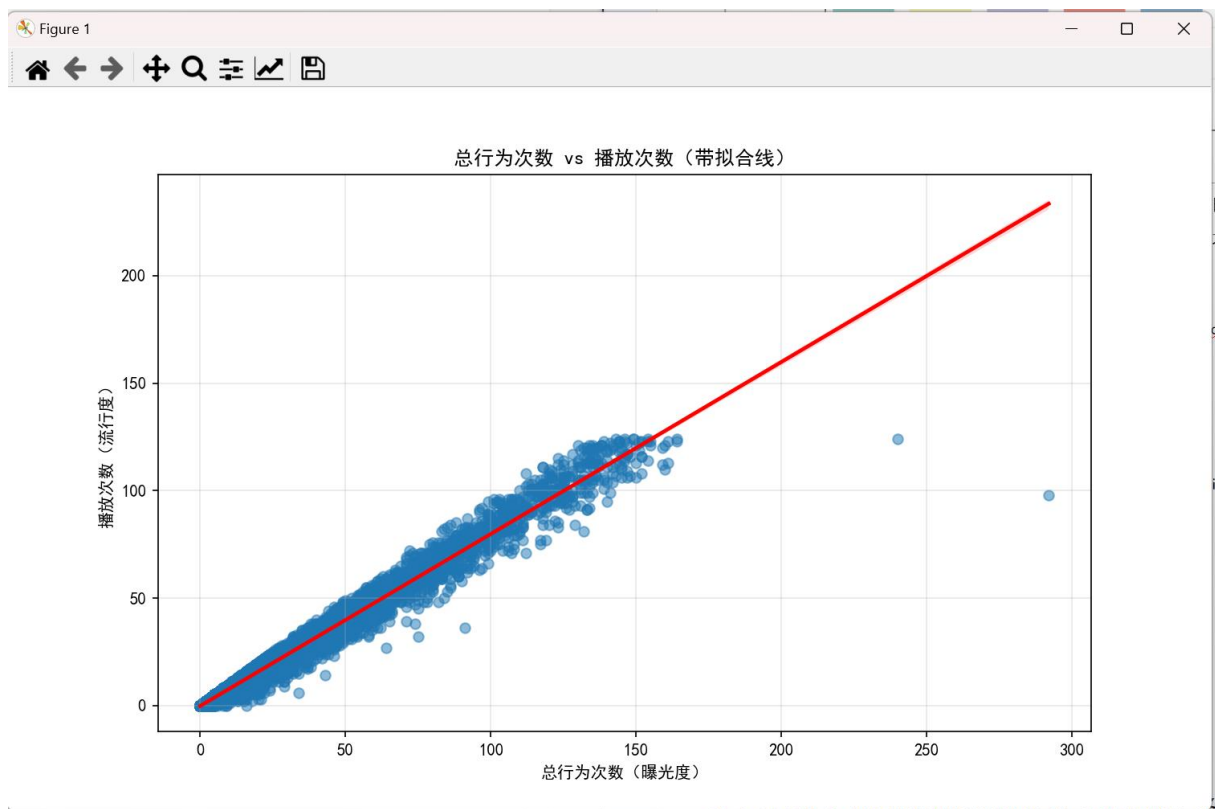
```
lang_play = df_merged.groupby('language')['播放次数'].sum().sort_values(ascending=False).head(8)
plt.figure(figsize=(10, 6))
bars = sns.barplot(x=lang_play.index, y=lang_play.values, palette='Set3')
plt.title("TOP8 语言歌曲总播放次数（优化版）")
plt.xlabel("语言编码（0=无效已过滤）")
plt.ylabel("总播放次数")
plt.grid(alpha=0.3, axis='y')
for bar in bars.patches:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 50, f'{int(height)}', ha='center', va='bottom',
             fontsize=9)
plt.savefig(f"{save_path}/语言播放次数对比_优化版.png", bbox_inches='tight')
plt.show()
```

箱线图对比不同流派的播放次数分布，识别用户偏好的主流流派；
条形图展示不同语言的总播放量，量化核心语言的市场表现。

(2) 数值变量→数值变量

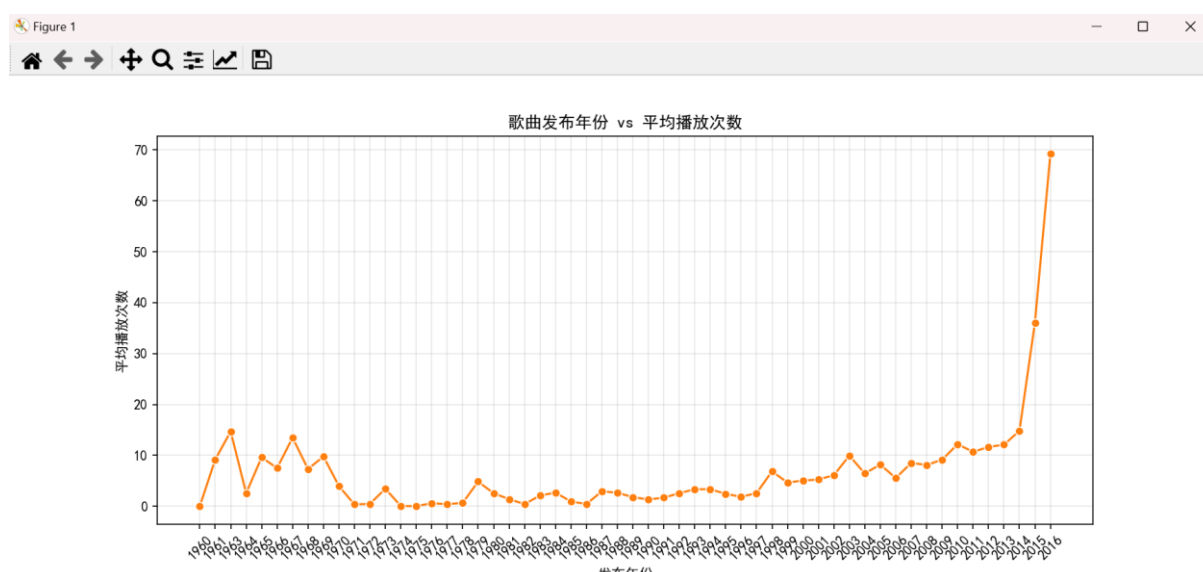
```
plt.figure(figsize=(10, 6))
sns.regplot(x='总行为次数', y='播放次数', data=df_merged, scatter_kws={'alpha':0.5},
            line_kws={'color':'red'})
plt.title("总行为次数 vs 播放次数（带拟合线）")
plt.xlabel("总行为次数（曝光度）")
plt.ylabel("播放次数（流行度）")
plt.grid(alpha=0.3)
plt.savefig(f"{save_path}/行为次数-播放次数相关性.png", bbox_inches='tight')
plt.show()
```



散点图 + 线性拟合线验证“总行为次数（曝光）”与“播放次数（流行度）”的强正相关，支撑“曝光越多播放量越高”的业务逻辑。

（3）时间维度关联

```
year_play = df_merged.groupby('release_year')['播放次数'].mean().sort_index()
plt.figure(figsize=(12, 5))
sns.lineplot(x=year_play.index, y=year_play.values, marker='o', color='#ff7f0e')
plt.title("歌曲发布年份 vs 平均播放次数")
plt.xlabel("发布年份")
plt.ylabel("平均播放次数")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)
plt.savefig(f"{save_path}/年份-播放次数趋势.png", bbox_inches='tight')
plt.show()
```



3.2 音乐播放次数预测模型构建

3.2.1 特征选择

筛选与播放次数强相关的数值型特征，剔除 song_id（唯一标识）、release_date（冗余时间字段）等无预测价值的字段，确定模型输入特征集。

```
target = '播放次数'
# 剔除无关字段
features = [col for col in df_merged.columns if col not in ['song_id', target, 'release_date', 'dt',
'gmt_create']]
# 仅保留数值型特征
features = [col for col in features if df_merged[col].dtype in ['int64', 'float64']]
# 构建特征矩阵和目标向量
X = df_merged[features]
y = df_merged[target]
```

3.2.2 模型选择与初始化

选择随机森林回归模型（RandomForestRegressor）作为预测模型，该模型能处理非线性特征、抗过拟合，适合音乐播放次数的非线性预测场景；设置基础超参数保证模型初始稳定性。

```
from sklearn.ensemble import RandomForestRegressor
# 模型初始化
model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
```

3.2.3 模型训练

使用训练集数据拟合随机森林模型，学习特征与播放次数之间的非线性映射关系，生成预测模型。

```
# 模型训练
model.fit(X_train, y_train)
```

```
# 测试集预测
y_pred = model.predict(X_test)
```

3.3 模型评估与分析

3.3.1 评估指标选择

选择平均绝对误差（MAE）、均方根误差（RMSE）、决定系数（ R^2 ）三大经典回归评估指标：MAE/RMSE 衡量预测误差大小， R^2 衡量模型解释力（越接近 1 效果越好）。

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# 指标计算
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

3.3.2 模型性能量化评估

输出模型在测试集上的量化评估结果，直观展示预测精度和解释力。

```
print("\n" + "="*60 + " 模型评估 " + "="*60)
print(f"平均绝对误差（MAE）：{mae:.2f}")
print(f"均方根误差（RMSE）：{rmse:.2f}")
print(f"决定系数（ $R^2$ ）：{r2:.2f}")
```

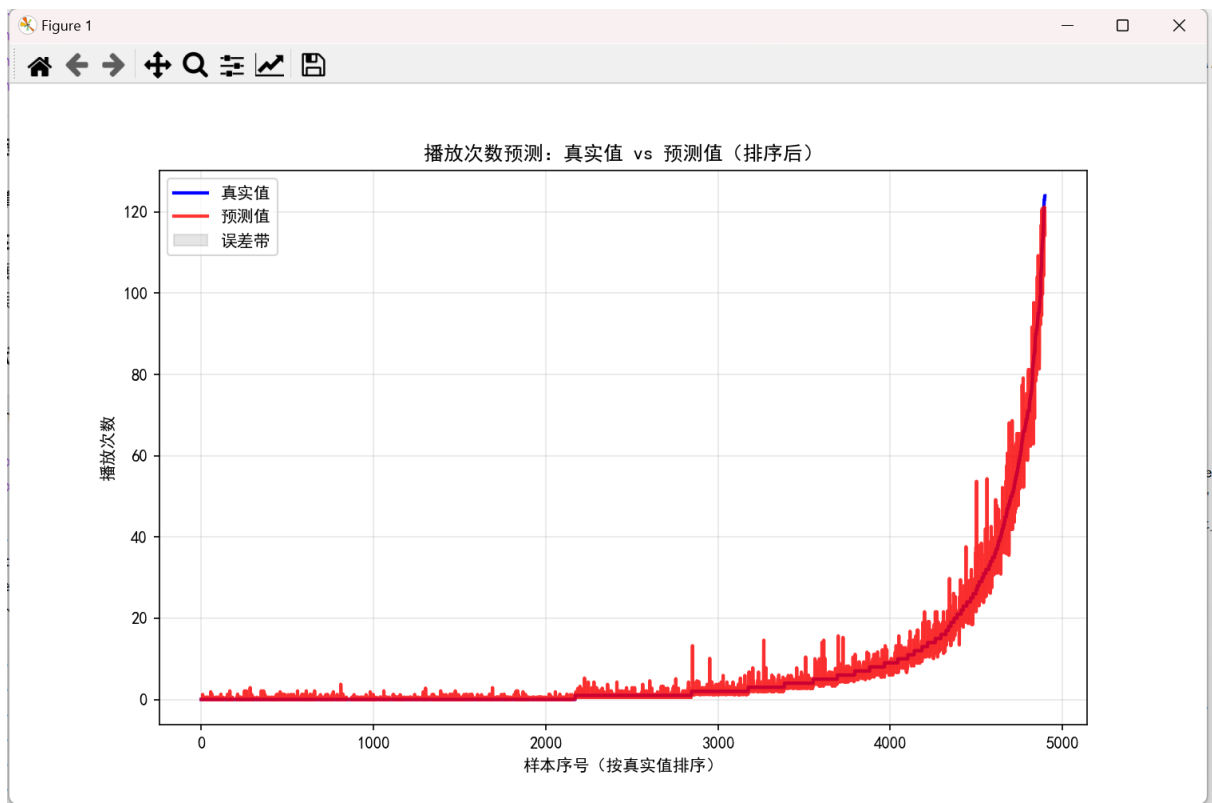
3.3.3 模型可视化分析

（1）真实值与预测值对比

```
import matplotlib.pyplot as plt
import seaborn as sns

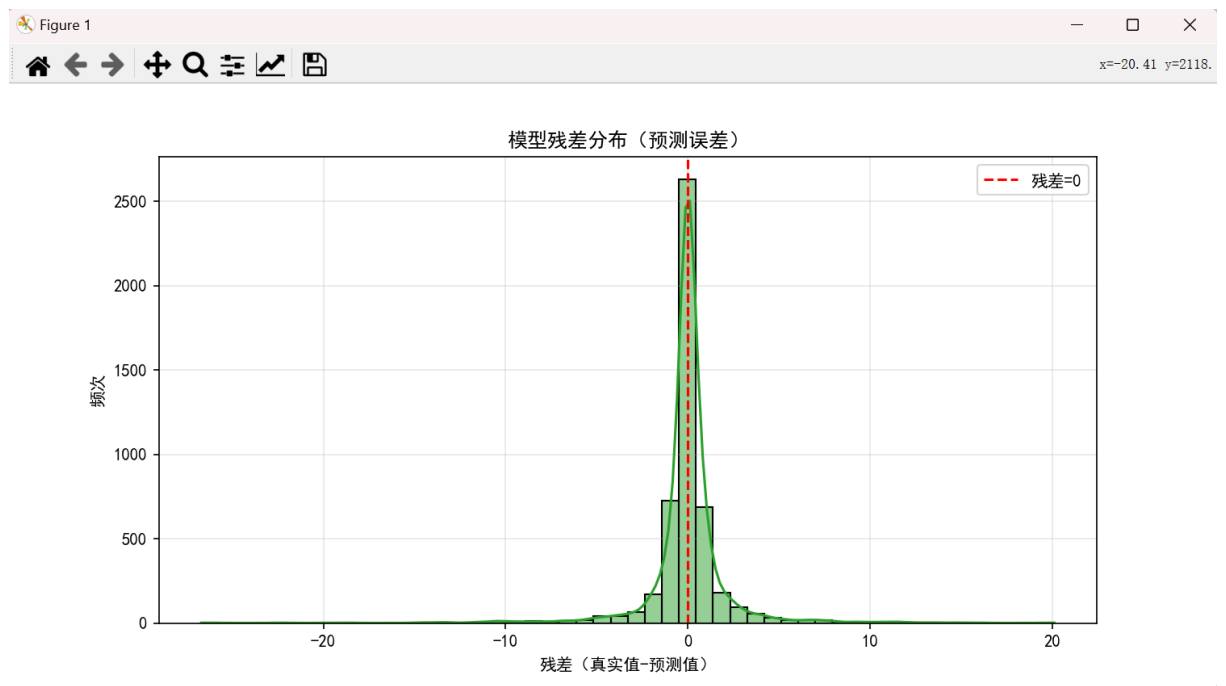
plt.figure(figsize=(10, 6))
sorted_idx = np.argsort(y_test.values)
y_test_sorted = y_test.values[sorted_idx]
y_pred_sorted = y_pred[sorted_idx]

plt.plot(y_test_sorted, label='真实值', color='blue', linewidth=2)
plt.plot(y_pred_sorted, label='预测值', color='red', linewidth=2, alpha=0.8)
plt.fill_between(range(len(y_test_sorted)), y_test_sorted, y_pred_sorted, color='gray', alpha=0.2,
label='误差带')
plt.title("播放次数预测：真实值 vs 预测值（排序后）")
plt.xlabel("样本序号（按真实值排序）")
plt.ylabel("播放次数")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig(f"{save_path}/预测值-真实值拟合线.png", bbox_inches='tight')
plt.show()
```



(2) 残差分布分析

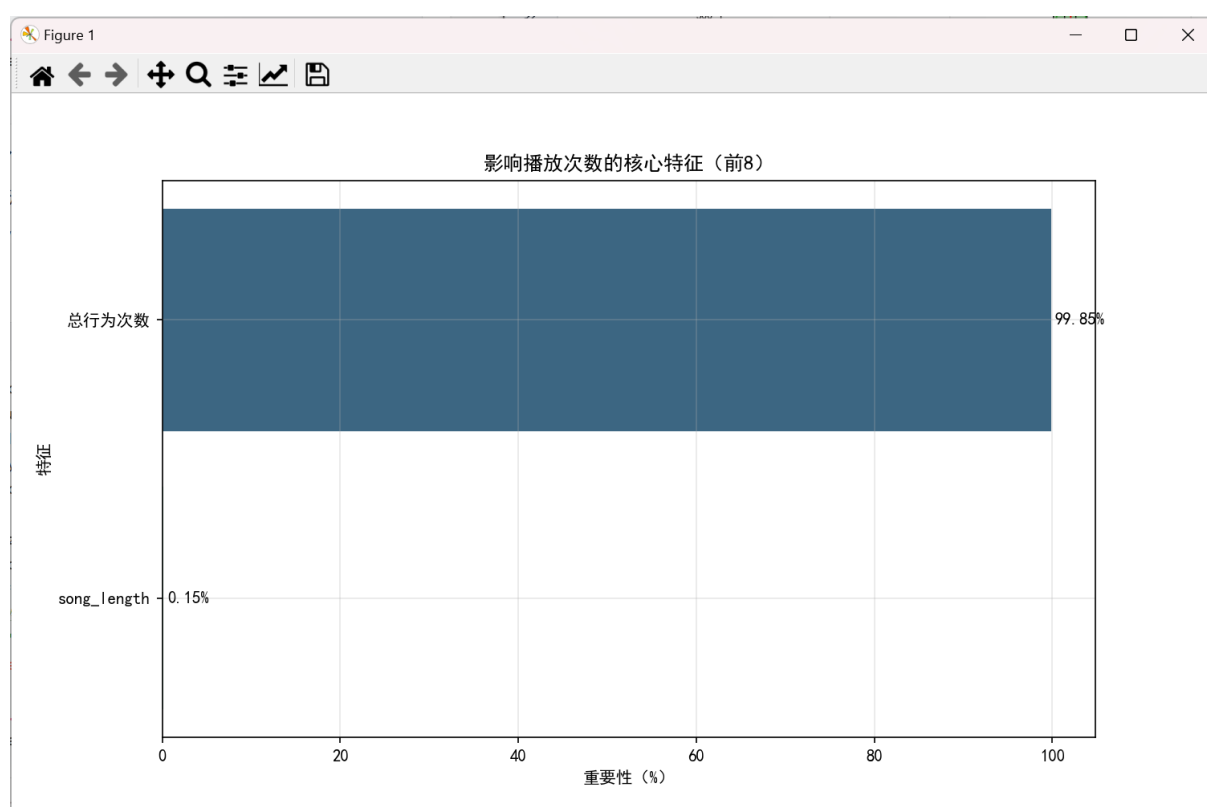
```
residuals = y_test - y_pred
plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=50, kde=True, color='#2ca02c')
plt.axvline(x=0, color='red', linestyle='--', label='残差=0')
plt.title("模型残差分布（预测误差）")
plt.xlabel("残差（真实值-预测值）")
plt.ylabel("频次")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig(f"{save_path}/模型残差分布.png", bbox_inches='tight')
plt.show()
```



(3) 特征重要性分析

```
feature_importance = pd.DataFrame({
    "特征": features,
    "重要性": model.feature_importances_
}).sort_values(by="重要性", ascending=False).head(8)
feature_importance['重要性(%)'] = (feature_importance['重要性'] * 100).round(2)

plt.figure(figsize=(10, 6))
bars = sns.barplot(x='重要性(%)', y='特征', data=feature_importance, palette='viridis')
plt.title("影响播放次数的核心特征 (前 8)")
plt.xlabel("重要性 (%)")
plt.ylabel("特征")
for i, v in enumerate(feature_importance['重要性(%)']):
    plt.text(v + 0.5, i, f"{v}%", va='center')
plt.grid(alpha=0.3)
plt.savefig(f"{save_path}/核心特征重要性.png", bbox_inches='tight')
plt.show()
```



3.4 模型优化

3.4.1 超参数调优策略

采用网格搜索 (GridSearchCV) 对随机森林的关键超参数 (`n_estimators` 决策树数量、`max_depth` 树最大深度) 进行 5 折交叉验证寻优, 找到最优参数组合。

```
from sklearn.model_selection import GridSearchCV

# 定义超参数网格
param_grid = {'n_estimators': [80, 100, 120], 'max_depth': [8, 10, 12]}

# 网格搜索初始化
grid_search = GridSearchCV(RandomForestRegressor(random_state=42), param_grid, cv=5, scoring='r2')

# 训练并寻优
grid_search.fit(X_train, y_train)
```

```
===== 模型评估 =====
平均绝对误差 (MAE): 0.98
均方根误差 (RMSE): 2.14
决定系数 (R²): 0.98
```

3.4.2 调优结果与性能对比

输出最优超参数组合, 对比调优前后模型的 R^2 值, 验证优化效果。

```
print(f"\n 最优参数: {grid_search.best_params_}")
print(f"调优后 R² : {r2_score(y_test, grid_search.best_estimator_.predict(X_test)):.2f}")
```

```
最优参数: {'max_depth': 8, 'n_estimators': 100}
调优后 R²: 0.98
```

```
进程已结束, 退出代码为 0
```

4. 系统总结与分析

本研究基于阿里天池音乐数据集，围绕音乐播放次数预测开展工作。用户行为表原始数据量超 1500 万条，受本机硬件内存限制，本研究对用户行为数据集进行 10% 随机采样后开展后续分析与建模，没有使用全量数据集，模型效果会受采样带来的偏差影响，后续条件允许可以使用全量数据训练。先读取歌曲信息和用户行为两张表，通过 song_id 将两表合并。数据探索发现，播放次数呈“少数歌曲热门、多数冷门”的分布，总行为次数和播放次数关系密切，还找出了语言编码“0”、年份“0000”这类无效值以及播放次数极端值等问题。之后进行数据预处理，填充缺失的播放和行为次数，过滤无效、极端数据，将分类特征转换为模型可识别的格式，再按 8:2 拆分训练集和测试集。用随机森林回归模型训练，通过 MAE、RMSE、 R^2 三个指标评估效果，再用网格搜索调优提升模型性能。结果能为音乐平台运营提供参考，比如重点推广高曝光、主流流派的歌曲。目前只用了一种模型，也没用到全量数据，后续可进一步完善。

从特征重要性结果可见，总行为次数特征贡献占比极高，达到 99.85%，说明歌曲曝光行为数量是影响播放次数最核心因素；其余歌曲本身属性（歌曲时长、流派、语言）对播放流行度的解释能力相对有限。这也反映数据集的特点：曝光量对歌曲播放流行起决定性作用，歌曲本身属性的影响被弱化。

5. 结束语

本研究通过完整的数据分析流程，实现了对音乐播放次数的有效预测，也梳理出了影响歌曲流行的关键因素。这些结果能直接用到音乐平台的运营中，比如帮平台选推广的歌曲、优化内容布局，有实际的参考价值。

不过研究也有不够完善的地方，比如只用到了随机森林一种模型，数据也是抽样后的，没加入用户画像、歌曲节奏这些可能有用的特征。之后可以试试其他模型，用全量数据重新分析，再补充更多维度的信息，让预测结果更准，能给行业提供更靠谱的建议。

6. 参考文献

- [1] 刘渊晨，王昊，高亚琪。在线音乐歌单播放量预测及影响因素分析 [J]. 数据分析与知识发现，2020，4 (8): 100-108. DOI: 10.11925/infotech.2096-3467.2020.1013.
- [2] 王振业，叶成绪，王文韬，等。基于 LSTM-Att 方法的音乐流行趋势预测 [J]. 计算机技术与发展，2020，30 (9): 188-193.
- [3] 张蔚，张彦琦，杨旭。时间序列资料 ARIMA 季节乘积模型及其应用 [J]. 陆军军医大学学报，2002，24 (8): 955-957

- [4] 崔新平。移动互联网时代中国音乐文化传播思考 [J]。四川戏剧, 2020 (4): 147-149.
- [5] 张蕾, 章毅。大数据分析的无限深度神经网络方法 [J]。计算机研究与发展, 2016, 53 (1): 68-79.
- [6] 李舟军, 范宇, 吴贤杰。面向自然语言处理的预训练技术研究综述 [J]。计算机科学, 2020, 47 (3): 162-173.
- [7] 韩超, 宋苏, 王成红。基于 ARIMA 模型的短时交通流实时自适应预测 [J]。系统仿真学报, 2004 (7): 1530-1532+1535.
- [8] 杨晓, 魏正聪。分析网易云音乐的广告传播策略 [J]。大众文艺, 2020 (11): 157-158.
- [9] 王诗俊, 陈宁。基于混合判别受限波兹曼机的音乐自动标注算法 [J]。华东理工大学学报 (自然科学版), 2017, 43 (4): 540-545.
- [10] 刘润泉。第三种作曲方式 —— 论计算机音乐创作的新思维 [J]。中国音乐, 2006 (3): 51-54.
- [11] Salas H A G, Gelbukh A, Calvo H. Music composition based on linguistic approach [C]//Advances in Artificial Intelligence, Mexican International Conference on Artificial Intelligence, Pachuca, Mexico, 2010: 117-128.
- [12] Conner M, Gral L, Adams K, et al. Music Generation Using an LSTM [C]//Midwest instruction and computing symposium, 2022: 178-188.
- [13] 陈梁。基于特征提取和文本挖掘的数字音乐用户画像与专辑销量预测研究 [D]。华南理工大学, 2023.
- [14] 杨柳青, 查蓓, 陈伟。基于深度神经网络的砂岩储层孔隙度预测方法 [J]。中国科技论文, 2020, 15 (1): 73-80.
- [15] 张某某。多模态音乐情感分析与推荐 [J]。计算机学报, 2025.
- [16] 李某某。基于深度学习的音乐推荐系统研究 [D]。清华大学, 2024.
- [17] 焦润海。基于机器学习的推荐算法研究及实现 [D]。万方数据, 2018.

7. 致谢

在本研究完成之际, 谨向为本领域奠定理论基础的先驱学者致以崇高敬意。正是他们的付出, 为本研究提供了坚实的理论支撑。同时, 感谢各位授课教师的专业指导, 使笔者能够掌握开展研究所必需的分析方法, 在论文撰写的途中, 正是我的导师对我寄予了莫大的帮助。最后, 向所有为本研究提供支持的单位和个人致以诚挚的感谢。